Deuteron · 3-minute video script

355 spoken words · 9 visual scenes · finish speaking by 2:55, then hold for five seconds. Record one clean, continuous take with no background music. Read only the narration below; the timecodes and visual cues are for rehearsal.

0:00–0:12 | Slide 1: The compiler transformations we built

Deuteron compiles scheduled Emmy Kernel IR into PTX for the RTX 5090. Our contribution is guarded compiler passes that change memory reuse, parallel work, and tensor pipelines.

Cue: Open on the presenter, then the title. Point to reuse, parallel work, and tensor pipelines.

0:12–0:27 | Slide 2: Where our passes run

We validate IR and metadata; each rewrite checks index and ownership conditions. Early passes vectorize loops, forward reduction inputs, reorder recurrences, and fold split-K grids. Specialized lowering follows; LLVM supplies optimization and NVPTX code generation.

Cue: Follow the pipeline. Say the LLVM credit clearly.

0:27–0:44 | Slide 3: Reuse data before loading it again

We look for repeated reads of the same input, or neighboring values loaded one at a time. We keep reusable values in registers and load nearby values together. That means fewer repeated trips to global memory.

Cue: Point to the repeated read, then show the saved values being reused.

Deuteron · reductions and tensors

0:44–1:03 | Slide 4: Reshape parallel work

We look for repeated calculations and blocks doing tiny pieces of work. We share dot products across threads, remove unnecessary reduction steps, and merge small pieces when enough parallel work remains. That reduces duplicated work and scheduling overhead.

Cue: Show repeated work becoming shared work, then small pieces being combined.

1:03–1:26 | Slide 5: Turn contractions into tensor work

We look for matrix multiplication written as ordinary multiply-and-add loops. We replace them with tensor-core operations, using three lower-precision products to approximate the result while adding them together in 32-bit precision. We also arrange shared memory so tensor cores can read it efficiently.

Cue: Follow the loop into tensor-core operations. Say that the result is approximate.

1:26–1:49 | Slide 6: Rewrite the pipeline feeding tensor cores

We look for tensor-core loops that repeatedly copy and convert their matrix inputs. For the supported BF16 path, we fetch upcoming inputs early and pack them before the tensor cores need them. We also skip the final prefetch when there is no more work.

Cue: Show the next inputs arriving early, then the unused final load being removed.

Deuteron · attention and results

1:49–2:12 | Slide 7: Recognize and stage the attention recurrence

We look for attention loops that repeat the score calculation across output columns. We calculate scores once per group and reuse their weights. We also load neighboring keys together and reuse temporary storage once its values are no longer needed.

Cue: Show one score calculation feeding several outputs, then temporary storage being reused.

2:12–2:30 | Slide 8: Redistribute probability–value work

We look for attention loops where each thread scans every key for one output column. We split keys among thread groups, load four adjacent columns together, and combine their partial results. Each column keeps its original normalization.

Cue: Show the keys split among groups, then their partial results being combined.

2:30–2:55 | Slide 9: Measure the whole compiler on RTX 5090

On fifteen open workloads, geometric mean warm latency is three point seven five microseconds. Speedups are one point six six times over generated CUDA with NVCC, and one point one five times over the tested Triton baseline. All two hundred seventy calls passed the original checks. These are whole-compiler results with a limited Triton search.

Cue: Slow for the ratios. Hold the results slide silently from 2:55 to 3:00.

2:55–3:00: stop speaking; hold slide 9. Leave extra silence on the raw recording for editing. The full recording setup, pronunciation notes, optional cuts, and edit plan are in the companion Markdown files.